

03 矩阵乘法

3.1 矩阵乘法第一视角

行列对齐、内积展开、向量内积与外积



鸢尾花书·《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>

本节你将掌握的核心技能

1

行列对齐理解乘法形状匹配： AB 相乘，将 A 行对齐放左侧，将 B 列对齐放上方，可直观判断 AB 的合法性及结果形状

2

内积视角计算元素：矩阵乘法中每个元素是 A 的一行与 B 的一列内积

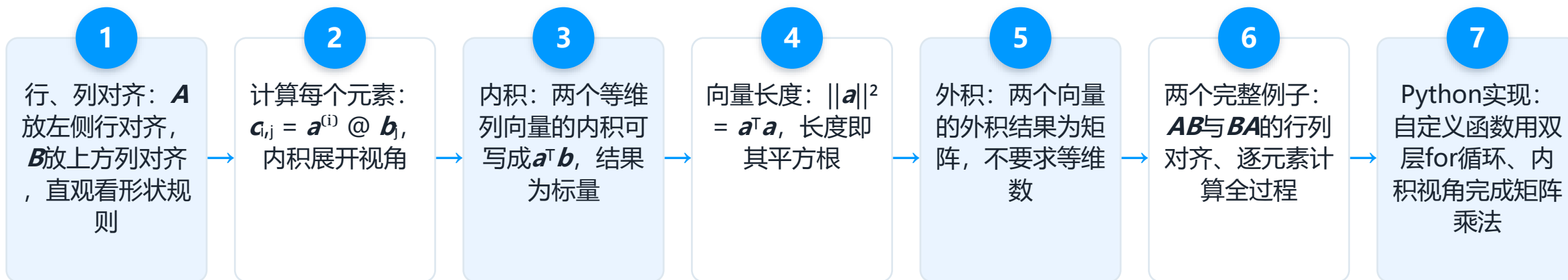
3

向量内积可转化为矩阵乘法： 1×1 矩阵相当于标量

4

向量外积生成矩阵：构造秩一矩阵，为下一节“矩阵乘法第二视角”做铺垫

本节知识脉络



换个视角，重新理解矩阵乘法

上一章已经介绍过矩阵乘法，本章专门介绍理解矩阵乘法的不同视角

- 矩阵乘法第一视角，通过将矩阵 A 和矩阵 B 以特定方式对齐，帮助我们理解乘法的形状要求
- 同时，这一视角还揭示了每个结果元素的计算过程——内积

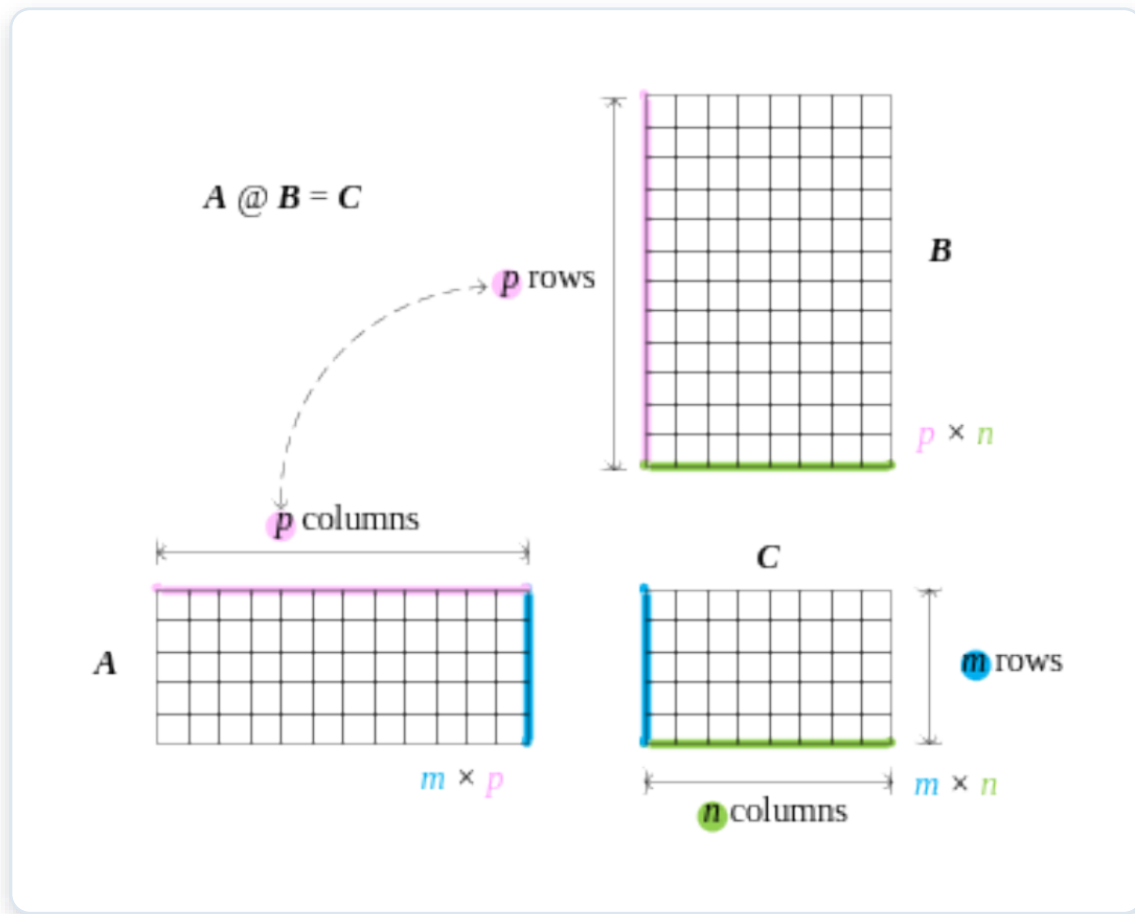
行、列对齐

矩阵**A**是 $m \times p$ 矩阵，矩阵**B**是 $p \times n$ 矩阵；将**A**放在结果**C**左侧行对齐，**B**放在**C**上面列对齐

- **A**放在**C**左侧，意味着**A**的行数 m 决定了**C**的行数 m
- **B**放在**C**上面，意味着**B**的列数 n 决定了**C**的列数 n ；因此**A** @ **B**结果**C**的形状是 $m \times n$

图1 • A放在AB的左侧，行对齐；B放在AB的上面，列对齐

- 这种对齐方式揭示了矩阵乘法的核心规则：**A**的列数必须等于**B**的行数，否则无法对齐
- **A**为 $m \times p$ ，**B**为 $p \times n$ ，中间维度 p 被“消去”，结果**C**的形状为 $m \times n$



计算AB的第i行、第j列元素

对齐后，**C**的每个元素 $c_{i,j}$ 是**A**的第i行与**B**的第j列对应元素相乘并求和的结果

- 这种空间对齐的视角不仅帮助我们理解矩阵乘法的形状，还让我们直观感受到两个矩阵的信息是如何整合在一起的
- 接下来看第i行、第j列元素究竟是怎样一步步算出来的

图2 • 计算 AB 的第 i 行、第 j 列元素

- A 的第 i 行（浅蓝到深蓝渐变）与 B 的第 j 列（浅橙到深橙渐变）对齐后，交汇处正是 c_{ij}
- A 放在 AB 的左侧，行对齐； B 放在 AB 的上面，列对齐

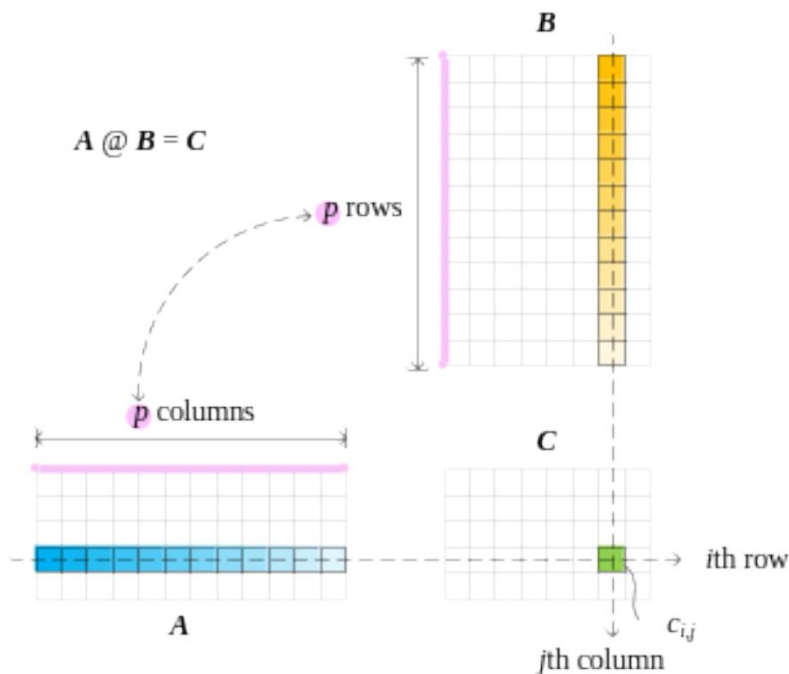


图 2. 计算 AB 的第 i 行、第 j 列元素： A 放在 AB 的左侧，行对齐； B 放在 AB 的上面，列对齐

矩阵乘法第一视角，也叫内积展开

$$c_{ij} = \mathbf{a}^{(i)} @ \mathbf{b}_j = \mathbf{a}^{(i)T} \cdot \mathbf{b}_j$$

- 矩阵 $\mathbf{A}$ 写成一组行向量，每个行向量记作 $\mathbf{a}^{(i)}$ ；矩阵 $\mathbf{B}$ 写成一组列向量，每个列向量记作 $\mathbf{b}_j$
- 把矩阵乘积 $\mathbf{C} = \mathbf{AB}$ 的第 i 行、第 j 列元素 c_{ij} 写成一个矩阵乘法，也可以用向量内积计算

图3 • 内积展开视角看矩阵乘积 $\mathbf{a}^{(i)} @ \mathbf{b}_j = c_{ij}$

- 行向量 $\mathbf{a}^{(i)}$ 有 p 列，列向量 $\mathbf{b}_j$ 有 p 行，二者对应元素相乘后求和，得到标量 c_{ij}
- 这一过程就是线性代数中的内积（inner product），也是Python中numpy.dot()函数的核心运算

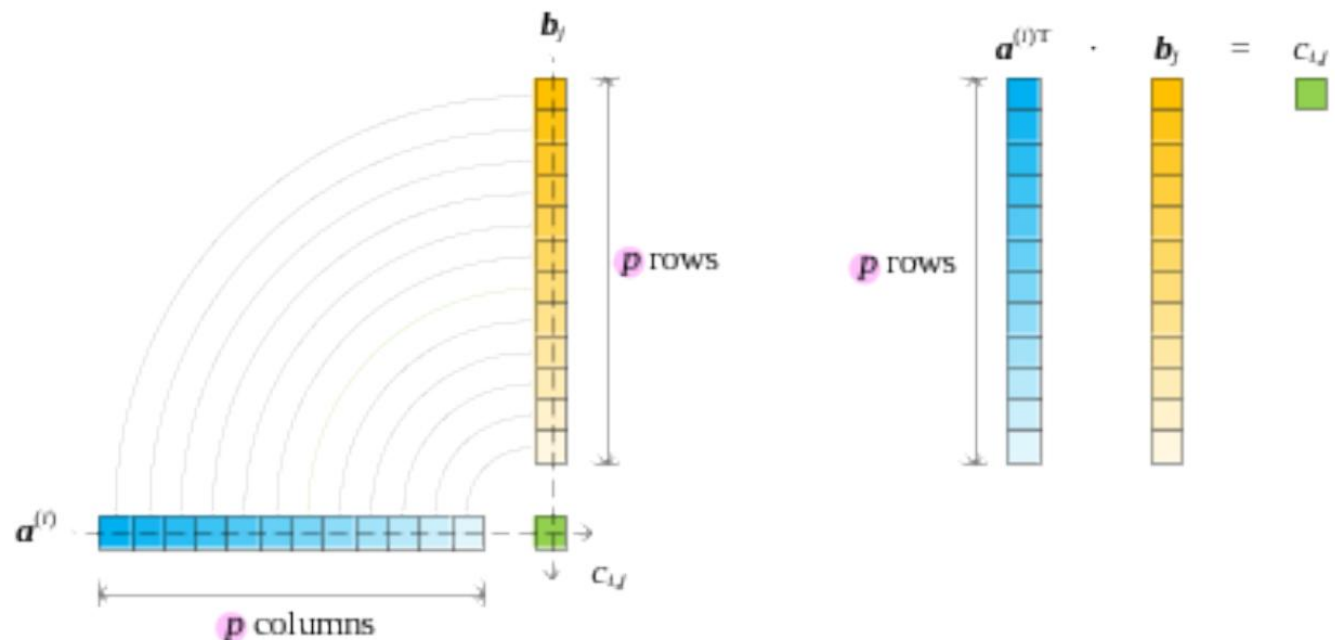


图 3. 内积展开视角看矩阵乘积 $\mathbf{a}^{(i)} @ \mathbf{b}_j = c_{ij}$

内积也可以写成矩阵乘法形式

$$n\text{维向量}\mathbf{a}、\mathbf{b}\text{的内积: } \mathbf{a} \cdot \mathbf{b} = \langle \mathbf{a}, \mathbf{b} \rangle = \mathbf{a}^T \mathbf{b} = \mathbf{b}^T \mathbf{a}$$

- 两个等维数的列向量内积，既可以写成点积符号，也可以写成矩阵乘法 $\mathbf{a}^T @ \mathbf{b}$ 或 $\mathbf{b}^T @ \mathbf{a}$
- 从矩阵形状看， $(1 \times n) @ (n \times 1)$ 夹在中间的 n 被“消去”，结果形状为 (1×1) ，即标量

图4 • 向量内积也可以写成矩阵乘法形式

- 给定 $\mathbf{a} = [1, 2, 3]^T$ 、 $\mathbf{b} = [4, 5, 6]^T$ ， $\mathbf{a}^T @ \mathbf{b} = [1 \ 2 \ 3] @ [4, 5, 6]^T = 32$ ， $\mathbf{b}^T @ \mathbf{a}$ 结果同样为 32
- 1×1 矩阵可以视作标量；无论用 $\mathbf{a}^T \mathbf{b}$ 还是 $\mathbf{b}^T \mathbf{a}$ ，结果完全一致

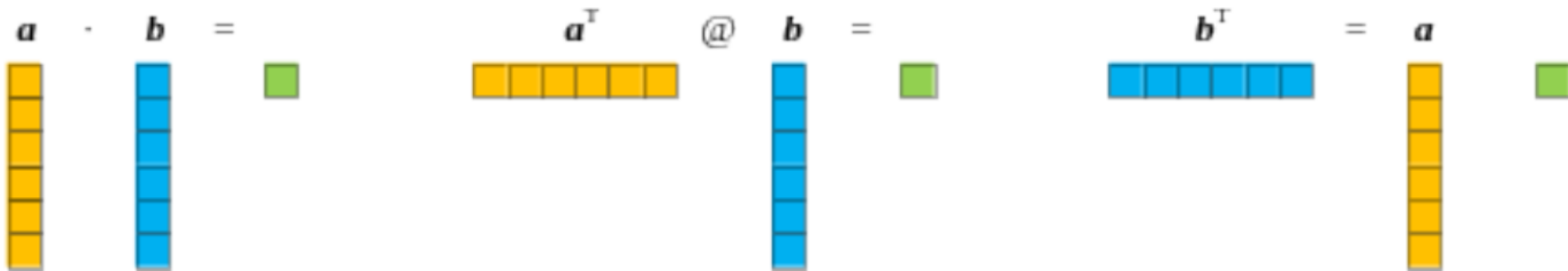


图 4. 向量内积也可以写成矩阵乘法形式

向量长度也能用矩阵乘法计算

$$\|\mathbf{a}\|^2 = \|\mathbf{a}\|_2^2 = \mathbf{a} \cdot \mathbf{a} = \langle \mathbf{a}, \mathbf{a} \rangle = \mathbf{a}^T \mathbf{a}, \quad \|\mathbf{a}\| = \sqrt{\mathbf{a}^T \mathbf{a}}$$

- 列向量 $\mathbf{a}$ 长度（大小、L2范数、模、欧几里得范数）的平方，既可以通过内积求得，也可以用矩阵乘法计算
- 给定 $\mathbf{a} = [1, 2, 3]^T$, $\|\mathbf{a}\| = \sqrt{([1 \ 2 \ 3] @ [1, 2, 3]^T)} = \sqrt{14}$

图5 • 向量长度的平方也可以写成矩阵乘法形式

- $a \cdot a$ 与 $a^T @ a$ 结果完全相同，都等于向量各分量平方和
- 这一关系在后续学习向量空间、投影、最小二乘时会反复用到

The diagram illustrates two equivalent ways to calculate the squared length of a vector a . On the left, the dot product $a \cdot a$ is shown, where a is represented by a vertical column of 6 yellow squares, followed by a dot, another a (another vertical column of 6 yellow squares), an equals sign, and a single green square representing the result. On the right, the matrix multiplication $a^T @ a$ is shown, where a^T is represented by a horizontal row of 6 yellow squares, followed by an '@' symbol, another a (a vertical column of 6 yellow squares), an equals sign, and a single green square representing the result.

图 5. 向量长度的平方也可以写成矩阵乘法形式

外积：内积的反面

两个向量的内积结果为标量，两个向量的外积（outer product）结果则是矩阵

- 外积 $\mathbf{a} @ \mathbf{b}^T$ 不要求 $\mathbf{a}$ 、 $\mathbf{b}$ 等维数： $\mathbf{a}$ 为 m 维列向量， $\mathbf{b}$ 为 n 维列向量，结果为 $m \times n$ 矩阵
- 这种特殊的矩阵乘法形态会帮助我们理解下一节的“矩阵乘法第二视角”与秩一矩阵

图6 • 向量a、b的两个外积

- $a @ b^T$ 把 a 的每个分量与 b 整体相乘，逐行“染色”生成矩阵； $b @ a^T$ 则反过来
- 两个外积结果互为转置： $(a @ b^T)^T = b @ a^T$

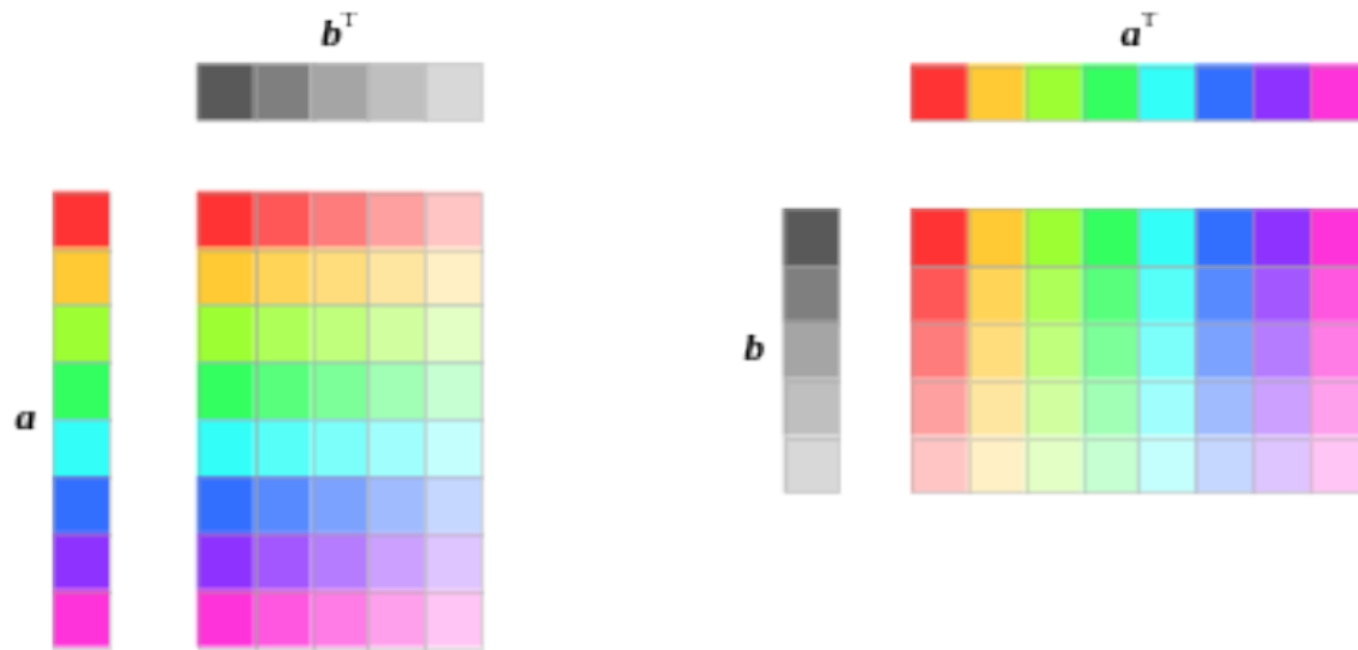


图 6. 向量 a 、 b 的两个外积

外积例子：形状可以不同

给定 $\mathbf{a} = [1, 2, 3, 4, 5]^T$ 、 $\mathbf{b} = [1, 2, 3]^T$ （不要求等维数）

- $\mathbf{a} @ \mathbf{b}^T$: $(5 \times 1) @ (1 \times 3)$ 夹在中间的1被“消去”，结果形状为 5×3
- $\mathbf{b} @ \mathbf{a}^T$: $(3 \times 1) @ (1 \times 5)$ 夹在中间的1被“消去”，结果形状为 3×5 ；两个结果互为转置

回到本书前文用过的例子： $A @ B$

$$A = [1,2,3; 4,5,6], \quad B = [1,4; 2,5; 3,6], \quad A @ B = [14,32; 32,77]$$

- A 写成两个行向量 $\mathbf{a}^{(1)}$ 、 $\mathbf{a}^{(2)}$ ， B 写成两个列向量 $\mathbf{b}_1$ 、 $\mathbf{b}_2$ 后， $A @ B$ 可以写成四个内积
- $\mathbf{c}_{1,1} = \mathbf{a}^{(1)} @ \mathbf{b}_1$, $\mathbf{c}_{1,2} = \mathbf{a}^{(1)} @ \mathbf{b}_2$, $\mathbf{c}_{2,1} = \mathbf{a}^{(2)} @ \mathbf{b}_1$, $\mathbf{c}_{2,2} = \mathbf{a}^{(2)} @ \mathbf{b}_2$

图7 • A放在AB的左侧，行对齐；B放在AB的上面，列对齐

- $a^{(1)} @ b_1$ 是形状为 1×3 的行向量与形状为 3×1 的列向量相乘，结果为 1×1 矩阵，相当于一个标量
- 四个内积 $c_{1,1}$ 、 $c_{1,2}$ 、 $c_{2,1}$ 、 $c_{2,2}$ 依次填入结果矩阵对应位置，即得 $A @ B = [14,32; 32,77]$

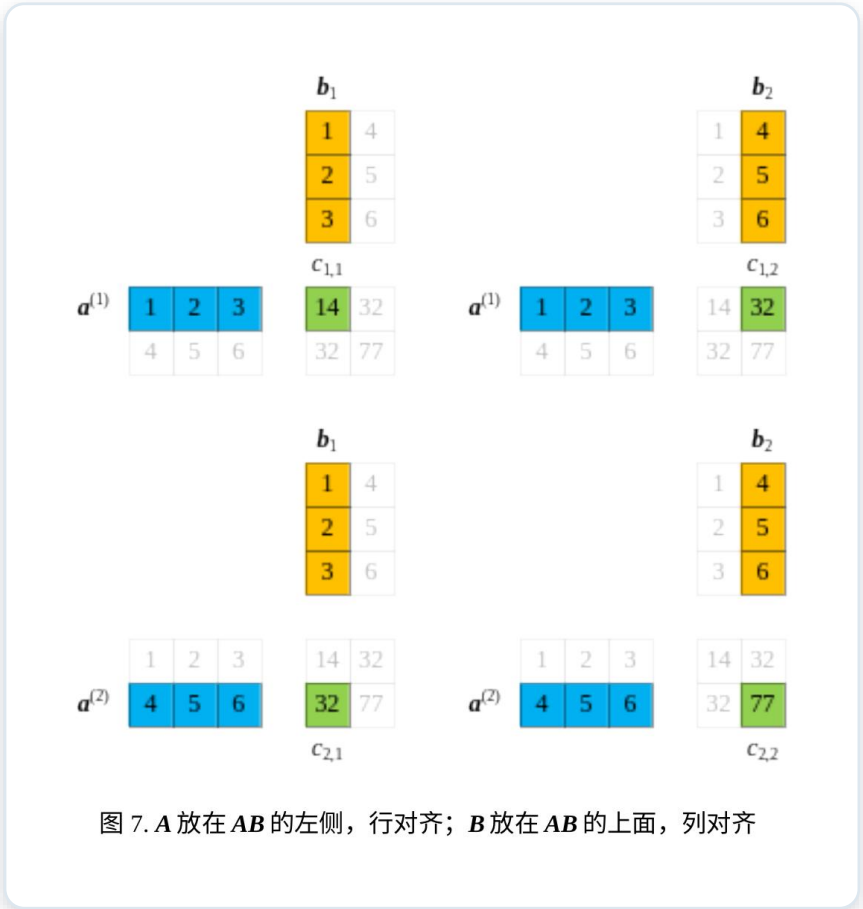


图 7. A 放在 AB 的左侧，行对齐；B 放在 AB 的上面，列对齐

第二个例子

再看第二个例子： $B @ A$

$$D = B @ A = [1,4; 2,5; 3,6] @ [1,2,3; 4,5,6] = [17,22,27; 22,29,36; 27,36,45]$$

- B 写成三个行向量 $\mathbf{b}^{(1)}$ 、 $\mathbf{b}^{(2)}$ 、 $\mathbf{b}^{(3)}$ ， A 写成三个列向量 $\mathbf{a}_1$ 、 $\mathbf{a}_2$ 、 $\mathbf{a}_3$ 后， $B @ A$ 可以写成九个内积
- 同一对矩阵，交换顺序后不仅结果不同，连结果矩阵的形状也从 2×2 变成了 3×3

九个内积拼成结果

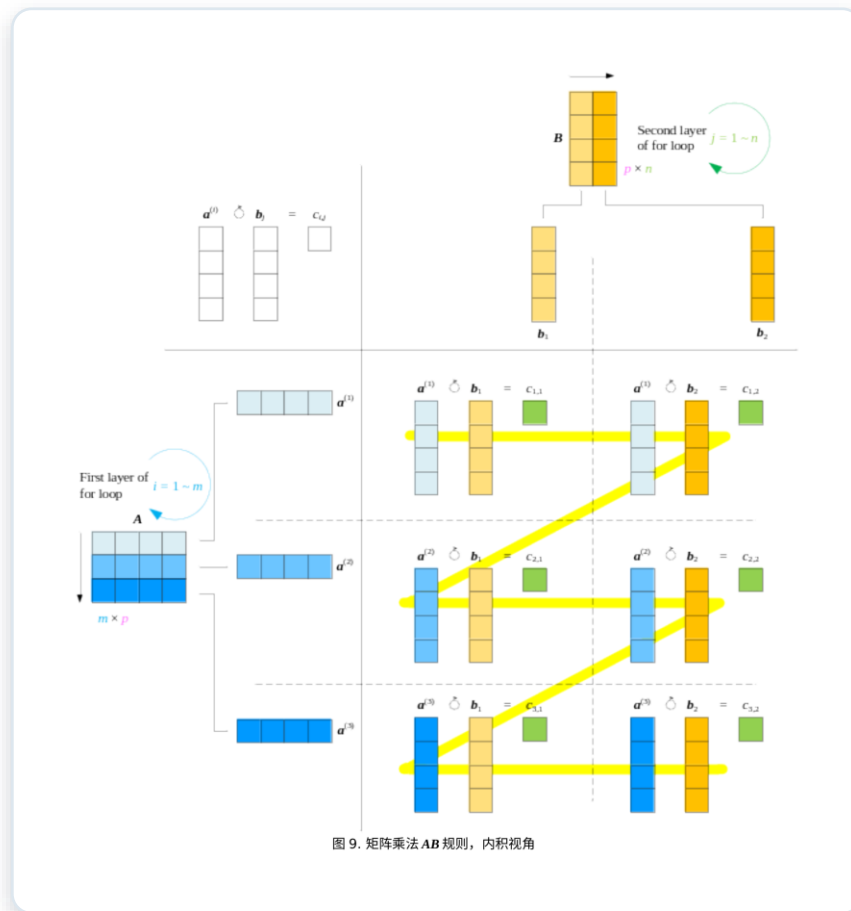
图8 · 计算每个元素：B放在BA的左侧，行对齐；A放在BA的上面，列对齐

- $d_{1,1}$ 、 $d_{1,2}$ $d_{3,3}$ 共九个内积，分别对应 $b^{(i)}$ 与 a 的组合
- 对比图7、图8可以看到，矩阵乘法第一视角同样适用于任意形状的 A 、 B ，只要满足列数等于行数



图9 • 矩阵乘法AB规则，内积视角

- 第一层循环遍历 A 的每一行 ($i = 1 \sim m$)，第二层循环遍历 B 的每一列 ($j = 1 \sim n$)
- 每完成一次 $a^{(i)} @ b_j$ ，就把结果填入 C 的第 i 行第 j 列，直至填满整个 $m \times n$ 矩阵



代码1 · 自定义Python函数 计算矩阵乘法

LA_03_01_02.ipynb

- 用def定义
matrix_multiplication_inner(A, B)函数
，先获取A、B形状，检测p_A是否等于
p_B，不等则报错
- 初始化结果矩阵C为(m, n)全零矩阵；双
层for循环遍历每一行i、每一列j
- 取出row_i_A (A的第i行)、col_j_B (B
的第j列)，用numpy.dot()计算内积，
赋值给C[i, j]

```
## 初始化
import numpy as np

## 自定义函数，矩阵乘法第一视角
def matrix_multiplication_inner(A, B):

    # 获取矩阵 A 和 B 的形状
    m, p_A = A.shape
    p_B, n = B.shape

    # 检测矩阵形状是否符合矩阵乘法规则
    if p_A != p_B:
        raise ValueError('Dimensions do not match')

    # 初始化结果矩阵 C，形状 (m, n)，初始值设为 0
    C = np.zeros((m, n))

    # 计算每个元素
    for i in range(m):
        for j in range(n):
            row_i_A = A[i, :]          # A 的第 i 行 (行向量)
            col_j_B = B[:, j]          # B 的第 j 列 (列向量)
            C[i, j] = np.dot(row_i_A, col_j_B) # 计算内积

    return C

## 矩阵乘法
A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = A.T

## 矩阵乘法
matrix_multiplication_inner(A, B)
matrix_multiplication_inner(B, A)
```

内积展开：理解矩阵乘法的第一把钥匙

行列对齐看形状，内积展开算元素——这是理解矩阵乘法最直观的第一视角

- 内积让两个向量“合二为一”得到标量，外积让两个向量“铺展开”得到矩阵，二者互为对偶
- 下一节将从外积出发，介绍矩阵乘法第二视角——用秩一矩阵之和理解 AB

本节关键术语

行列对齐 *Row-Column Alignment* — A 行对齐放左侧, B 列对齐放上方, 直观判断 AB 形状

内积 *Inner Product* — $a \cdot b = a^T b = b^T a$, 两个等维向量的乘积为标量

秩一矩阵 *Rank-One Matrix* — 向量外积生成的矩阵, 下一节展开讲解

形状规则 $A @ B$ 合法的前提: A 的列数必须等于 B 的行数

行向量 / 列向量 A 写成行向量组 $a(i)$, B 写成列向量组 b_j

内积展开 矩阵乘法第一视角, $c_{ij} = a(i) @ b_j$, 逐元素计算

外积 *Outer Product* — $a @ b^T$, 两个向量的乘积为矩阵, 不求等维数

向量长度 $\|a\| = \sqrt{a^T a}$, 也叫L2范数、模、欧几里得范数

标量 1×1 矩阵可以视作标量

双层for循环 外层遍历行号 i , 内层遍历列号 j , 逐一计算 $C[i,j]$

配套代码与延伸阅读

延伸阅读 • LA_03_01_01.ipynb

以矩阵乘法第一视角计算本节全部例题，建议自学并逐行调试

延伸阅读 • LA_03_01_02.ipynb

自定义Python函数计算矩阵乘法，内积视角实现

配套内容 • 矩阵乘法第二视角

下一节将从外积出发，用秩一矩阵之和理解AB

开源仓库

全书代码与图像素材：<https://github.com/Visualize-ML>

习题 Q1 – Q9

Q1 给定列向量 $a=[1,2]^T$ ，用矩阵乘法第一视角方法摆放并计算 $a^T @ a$ 、 $a @ a^T$ 。

Q3 给定 $A=[1,2;3,4]$ 和 $b=[1,2]^T$ ，用矩阵乘法第一视角方法摆放并计算 $A @ b$ 、 $b^T @ A^T$ 。

Q5 修改代码1，调换内外两层for循环，验证结果是否改变。

Q7 写一个函数`outer_product_manual(a, b)`，用双层for循环（不调用`np.outer`）计算 $a @ b.T$ ，并与`np.outer(a, b)`结果比对。

Q9 对若干随机生成的向量，验证`np.linalg.norm(a)`与`np.sqrt(a.T @ a)`结果是否一致。

Q2 给定列向量 $a=[1,2]^T$ 和 $b=[1,2,3]^T$ ，用矩阵乘法第一视角方法摆放并计算 $a @ b^T$ 、 $b @ a^T$ 。

Q4 给定 $A=[1,2;3,4;5,6]$ ，用矩阵乘法第一视角方法摆放并计算 $A @ A^T$ 、 $A^T @ A$ 。

Q6 写一个函数`inner_product_via_matmul(a, b)`，用 $a.T @ b$ 计算向量内积，并与`np.dot(a, b)`结果比对。

Q8 用Python复现图7：给定第一个例子中的A、B，逐一打印 $a(1)@b1$ 、 $a(1)@b2$ 、 $a(2)@b1$ 、 $a(2)@b2$ 四个内积的计算过程。

习题 Q10 – Q18

Q10 写一个函数`check_shape_match(A, B)`，在做矩阵乘法前检测A的列数是否等于B的行数，模仿代码1中的形状检测逻辑。

Q12 构造一个5维列向量a和一个3维列向量b，用Python计算 $a @ b^T$ 与 $b @ a^T$ ，比较两者形状与本节结论是否一致。

Q14 用time模块比较`matrix_multiplication_inner(A, B)`与NumPy内置 $A @ B$ 在大规模矩阵（如 200×200 ）上的运行时间差异。

Q16 给定三个同维数向量a、b、c，验证 $(a^T @ b) * c$ 与 $a * (b^T @ c)$ 是否相等，并从形状角度解释背后的原因。

Q18 验证第二个例子中 $D = B @ A$ 是否满足 $D = D^T$ （对称矩阵），并解释当 $B = A^T$ 时为什么BA一定对称。

Q11 给定 $A=[1,2,3;4,5,6]$ 、 $B=A^T$ ，分别计算 $A @ B$ 、 $B @ A$ ，并解释为什么两者形状不同。

Q13 写一个函数`rank_one_check(M)`，判断给定矩阵M的所有行是否都是同一行向量的倍数（即M是否可能是某个外积）。

Q15 修改代码1，用纯NumPy向量化写法（不使用显式for循环）实现矩阵乘法，并验证结果一致。

Q17 写一个函数`visualize_alignment(A, B)`，仿照图1打印A、B、C三者的形状（ $m \times p$ 、 $p \times n$ 、 $m \times n$ ）。

习题 Q19 – Q25

Q19 对一个随机 4×2 矩阵 A ，计算 $A @ A^T$ 与 $A^T @ A$ ，验证两者是否都是对称矩阵。

Q21 用`matrix_multiplication_inner`验证Q5中内外循环调换后，结果矩阵是否完全相同，只是计算顺序不同。

Q23 写一个函数`count_multiplications(A, B)`，返回矩阵乘法 $A @ B$ 所需的标量乘法总次数 $m \times n \times p$ ，并与你实现的双层循环中的实际运算次数核对。

Q25 结合本节所学，写一个小程序：输入任意两个矩阵，先用行列对齐方法（图1风格）判断形状是否兼容，兼容则用内积视角计算并打印乘积。

Q20 写一个函数比较 $\text{trace}(A @ A^T)$ 与 A 中所有元素平方和（Frobenius范数的平方），验证二者是否相等。

Q22 构造一个 3×1 列向量 a 和一个 1×3 行向量 b^T ，计算 $a @ b^T$ ，并用`np.linalg.matrix_rank()`验证结果矩阵的秩是否为1。

Q24 在NumPy中尝试用形状不匹配的矩阵做乘法（如 2×3 与 2×3 相乘），捕获并打印错误信息，并与代码1中`raise`的`ValueError`做比较。

10 个可深入探讨的 Prompt (1/2)

- 1 请解释为什么矩阵乘法 $A @ B$ 要求 A 的列数等于 B 的行数，用行列对齐的方式说明这一规则的几何/空间直觉。
- 2 请证明矩阵乘积 $C = AB$ 的第 i 行第 j 列元素 c_{ij} 等于 A 第 i 行与 B 第 j 列的内积。
- 3 请解释为什么两个等维向量的内积 $a^T b$ 与 $b^T a$ 结果相同，而两个向量的外积 ab^T 与 ba^T 结果一般不同（除非互为转置关系外）。
- 4 请解释向量长度 $\|a\|$ 为什么可以写成 $\sqrt{a^T a}$ ，并说明这一关系与勾股定理之间的联系。
- 5 请解释为什么外积 $a @ b^T$ 不要求 a 、 b 维数相同，而内积 $a^T @ b$ 却要求维数相同。

10 个可深入探讨的 Prompt (2/2)

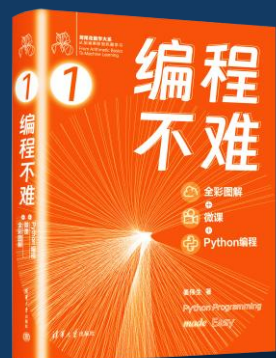
- 6 请解释为什么代码1中双层for循环调换顺序（先列后行）不会改变矩阵乘法的最终结果。
- 7 请解释为什么 $B @ A$ 的形状可能与 $A @ B$ 完全不同，并举例说明何时两者形状恰好相同。
- 8 请解释外积 $a @ b^T$ 生成的矩阵为什么一定是“秩一矩阵”，这对下一节理解矩阵乘法第二视角有什么帮助？
- 9 请解释矩阵乘法第一视角（内积展开）与用NumPy向量化实现矩阵乘法之间的关系，为什么向量化通常更快？
- 10 请举例说明矩阵乘法的内积视角在机器学习中的实际应用，比如全连接层的前向传播是如何用内积逐元素计算的？

数学

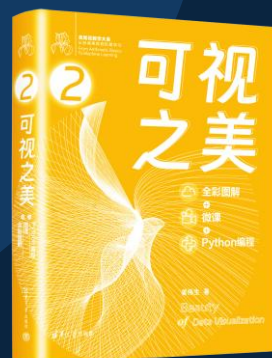
数学基础

线性代数

概率统计

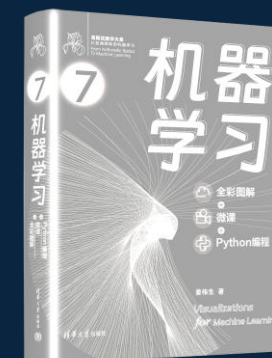
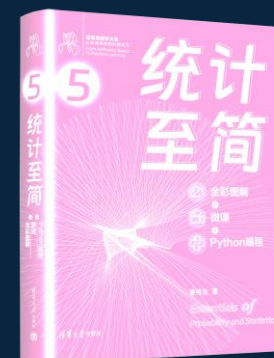


Python编程



数据可视化

工具



数据分析
图和网络

回归、降维
分类、聚类

实践

